

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

MIPS32 Assignment 6

August 2026

Instructions: *This is a group assignment. Each group must demonstrate the working programs to a Teaching Assistant for evaluation. The assignment must be submitted as a group submission through Moodle. Submit a single zipped folder named `MIPSAssignment6_{Group_Number}.zip`. The folder must contain the complete and well-commented MIPS32 assembly source files for both questions. Each source file must include a header containing the group number, the names and roll numbers of all group members, the assignment number, question number, and date of submission.*

For both questions, the program must contain a `main` section and the separate procedures specified in the question. The `main` section must perform input collection, initial validation, procedure invocation, final result printing, error-message printing, and program termination. The required computation must be performed inside the specified procedures.

Every non-leaf procedure must use the runtime stack correctly. In particular, it must preserve `$ra`, preserve every argument or intermediate value needed after another procedure call, restore the stack pointer before returning, and follow the standard MIPS32 register-usage conventions.

Question 1: Postfix Expression Evaluation Using an Explicit Stack

1. Objective

Write a MIPS32 assembly-language program that evaluates an arithmetic expression written in postfix notation.

The program must use an explicit operand stack stored in the `.data` segment. The operand stack must not be implemented using the runtime call stack.

The main evaluation must be implemented in a procedure named

`evaluate_postfix.`

The program must also contain separate procedures named

`push,` `pop,` `apply_operator.`

2. Problem Definition

In postfix notation, an operator appears after its two operands. For example,

$$(5 + 3) \times 2$$

is written as

5 3 + 2 *

and evaluates to

16.

For every binary operator, the right operand is popped first and the left operand is popped second. Thus, for the postfix fragment

8 3 -

the result must be

$8 - 3 = 5$,

not $3 - 8$.

The expression format for this assignment is restricted as follows:

- each operand is a single decimal digit from 0 to 9;
- valid operators are +, -, *, and /;
- every operand and operator must be separated from the next token by one or more spaces;
- the total input length, excluding the terminating null byte, must not exceed 80 characters; and
- all required test cases will produce results representable as signed 32-bit integers.

Integer division must follow the MIPS32 signed-integer division semantics. Division by zero must be detected and reported as an error.

3. Explicit Stack Representation

The operand stack must be stored as an array of 20 words:

<code>operand_stack: .space 80</code> <i># 20 signed integer entries</i>
--

A stack-top index must be maintained using the following convention:

`top = -1`

means that the stack is empty, and

`top = 0`

means that the first stack element is occupied.

The stack element at index i is located at

`operand_stack + 4i`.

The explicit operand stack and its top index must not be confused with the MIPS runtime stack addressed through `$sp`.

4. Required Procedure Interfaces

push

Item	Requirement
Procedure name	push
Input	\$a0 contains the operand-stack base address \$a1 contains the current top index \$a2 contains the stack capacity, which is 20 \$a3 contains the value to be pushed
Output	\$v0 contains the updated top index \$v1=0 on success and \$v1=1 on overflow
Return	The procedure must return using jr \$ra

pop

Item	Requirement
Procedure name	pop
Input	\$a0 contains the operand-stack base address \$a1 contains the current top index
Output	On success, \$v0 contains the popped value and \$v1 contains the updated top index On underflow, \$v1 must contain -2 ; \$v0 is then undefined
Return	The procedure must return using jr \$ra

apply_operator

Item	Requirement
Procedure name	apply_operator
Input	\$a0 contains the left operand \$a1 contains the right operand \$a2 contains the ASCII code of the operator
Output	\$v0 contains the arithmetic result when successful \$v1=0 on success \$v1=1 for division by zero \$v1=2 for an invalid operator
Return	The procedure must return using jr \$ra

evaluate_postfix

Item	Requirement
Procedure name	evaluate_postfix
Input	\$a0 contains the postfix-expression address \$a1 contains the operand-stack base address \$a2 contains the stack capacity, which is 20
Output	\$v0 contains the final result when successful \$v1=0 on success \$v1=1 for a malformed expression \$v1=2 for an invalid character or token \$v1=3 for stack overflow \$v1=4 for division by zero
Invocation	The procedure must be called using jal evaluate_postfix
Return	The procedure must return using jr \$ra

The evaluate_postfix procedure is a non-leaf procedure because it invokes push, pop, and apply_operator.

5. Algorithm

Algorithm 1 evaluate_postfix using an explicit operand stack

Require: A null-terminated postfix expression E

Ensure: Return the evaluated result or an error status

```

1:  $top \leftarrow -1$ 
2: Scan the expression one character at a time
3: while the null terminator has not been reached do
4:   if the current character is a space then
5:     Skip it
6:   else if the current character is a digit then
7:     Convert the ASCII digit to its integer value
8:     PUSH(value)
9:     if the stack overflows then
10:      Return stack-overflow status
11:   else if the current character is a valid operator then
12:      $right \leftarrow \text{POP}$ 
13:      $left \leftarrow \text{POP}$ 
14:     if either pop operation underflows then
15:      Return malformed-expression status
16:      $result \leftarrow \text{APPLY\_OPERATOR}(left, right, operator)$ 
17:     if division by zero is detected then
18:      Return division-by-zero status
19:     PUSH(result)
20:   else
21:     Return invalid-token status
22: if the stack contains exactly one value then
23:   Return that value
24: else
25:   Return malformed-expression status

```

6. Assignment Task

Write a MIPS32 assembly-language program that performs the following operations:

1. Ask the user to enter a postfix expression.
2. Read the expression using syscall 8 into a buffer of at least 82 bytes.
3. Remove or replace the newline inserted by syscall 8.
4. Reject an empty expression.
5. Initialize the explicit operand stack as empty.
6. Place the expression address in `$a0`.
7. Place the operand-stack address in `$a1`.
8. Place the capacity 20 in `$a2`.
9. Call `evaluate_postfix` using `jal evaluate_postfix`.
10. Use `push` and `pop` for all explicit stack operations.
11. Use `apply_operator` for all arithmetic operations.
12. Return the final result and status through `$v0` and `$v1`.
13. Print the result from `main` when evaluation succeeds.
14. Print an appropriate error message from `main` when evaluation fails.
15. Terminate the program using syscall 10.

7. Required Runtime-Stack Behaviour

Because `evaluate_postfix` invokes other procedures, it must:

1. allocate a runtime stack frame before the first nested procedure call;
2. save `$ra`;
3. preserve the expression pointer, explicit-stack address, current top index, current operator, and any operands needed across procedure calls;
4. restore all saved values;
5. deallocate the complete runtime stack frame; and
6. return using `jr $ra`.

The explicit operand stack must remain in the `.data` segment. The runtime stack may be used only for preserving procedure-call state.

8. Suggested Control-Flow Organization

```
.data
    expression:    .space 82
    operand_stack: .space 80

    # Input prompt
    # Result message
    # Invalid-expression message
    # Invalid-token message
    # Stack-overflow message
    # Division-by-zero message
    # Newline string

.text
.globl main

main:
    # Read the postfix expression
    # Remove the trailing newline
    # Reject an empty input
    # Pass addresses and capacity to evaluate_postfix

    jal evaluate_postfix

    # Examine the status returned in $v1
    # Print either the result or the appropriate error
    # Terminate using syscall 10

evaluate_postfix:
    # Input:  $a0 = expression address
    #         $a1 = explicit stack address
    #         $a2 = capacity
    # Output: $v0 = result when successful
    #         $v1 = status code

    # Scan tokens
    # Call push, pop, and apply_operator
    # Verify that exactly one result remains

    jr $ra

push:
    # Push one signed word onto the explicit stack
    jr $ra

pop:
    # Pop one signed word from the explicit stack
    jr $ra

apply_operator:
    # Apply +, -, *, or /
    jr $ra
```

The listing specifies only the required organization. Students must complete the MIPS32 instructions independently.

9. Input Validation and Error Conditions

The program must detect all of the following conditions:

- an empty expression;
- an invalid character or token;
- an operator encountered when fewer than two operands are available;
- more than one operand remaining after the complete expression has been processed;
- no operand remaining after processing;
- explicit-stack overflow; and
- division by zero.

Messages similar to the following may be used:

Error: malformed postfix expression. Error: invalid token in the expression. Error: explicit operand stack overflow. Error: division by zero.
--

10. Allowed Instructions and Directives

The solution may primarily use:

- `.data`, `.text`, `.globl`, `.ascii`, and `.space`;
- `la`, `li`, and `move`;
- `add`, `addi`, `addu`, `addiu`, `sub`, and `subu`;
- `mul`, or `mult` followed by `mflo`;
- `div` followed by `mflo`;
- `sll`;
- `lb`, `lbu`, `sb`, `lw`, and `sw`;
- `slt`, `slti`;
- `beq`, `bne`, `bgt`, `bge`, `blt`, `ble`, `bltz`, and `bgez`;
- `b`;
- `jal`;
- `jr`; and
- `syscall`.

Equivalent pseudo-instructions supported by MARS or QtSPIM may be used.

12. Sample Runs

Sample Run 1

```
Enter a postfix expression: 5 3 + 2 *  
The evaluated result is: 16
```

Sample Run 2

```
Enter a postfix expression: 8 3 - 2 /  
The evaluated result is: 2
```

Sample Run 3

```
Enter a postfix expression: 2 3 4 * +  
The evaluated result is: 14
```

Sample Run 4: Malformed Expression

```
Enter a postfix expression: 5 +  
Error: malformed postfix expression.
```

Sample Run 5: Division by Zero

```
Enter a postfix expression: 8 0 /  
Error: division by zero.
```

13. Required Test Cases

Postfix expression	Expected output
5 3 + 2 *	16
9 4 -	5
2 3 4 * +	14
8 3 - 2 /	2
7 2 /	3
9 5 2 + *	63
5 +	Malformed-expression error
5 4	Malformed-expression error
8 0 /	Division-by-zero error
2 A +	Invalid-token error
Empty input	Malformed-expression error

Additional test cases may be used during evaluation.

14. Correctness Requirements

A correct implementation must satisfy the following conditions:

1. The postfix expression is read into a character buffer.
2. The explicit stack is stored as an array of words in the `.data` segment.

3. The empty-stack top value is -1 .
4. Every operand is pushed using `push`.
5. Every operand retrieval is performed using `pop`.
6. For an operator, the right operand is popped before the left operand.
7. Arithmetic is performed using `apply_operator`.
8. The procedure detects underflow, overflow, invalid tokens, and division by zero.
9. Exactly one value remains after a valid expression is processed.
10. `evaluate_postfix` preserves `$ra` and all required state across nested procedure calls.
11. The runtime stack and explicit operand stack are used for their distinct intended purposes.
12. The result and status are returned through `$v0` and `$v1`.
13. The final output is printed by `main`.
14. The program terminates using `syscall 10`.

15. Submission Requirement for Question 1

The source file for Question 1 must be named:

`postfix_evaluator_{Group_Number}.S`

The file must begin with a comment header similar to:

```
# -----
# Group Number:
# Group Members (Name and Roll Number):
# 1.
# 2.
# 3.
# Course: Computer Organization and Architecture Laboratory
# Assignment: MIPS32 Assignment 6
# Question: 1
# Problem: Postfix expression evaluation using an explicit stack
# Date:
# -----
```

Question 2: Run-Length Encoding and Decoding

1. Objective

Write a MIPS32 assembly-language program that compresses a string using run-length encoding, decodes the generated representation, and verifies that the decoded string is identical to the original input.

The encoding and decoding operations must be implemented in separate procedures named

`encode_rle` and `decode_rle`.

The final comparison must be performed in a separate procedure named

`compare_strings`.

2. Problem Definition

Run-length encoding replaces each maximal run of repeated characters by the character followed by the decimal number of repetitions.

For example,

AAAABBCCCCC

must be encoded as

A4B2C5.

A run containing more than nine characters must use a multi-digit decimal count. For example,

AAAAAAAAAAAAA

must be encoded as

A12.

For this assignment, the original input string may contain only uppercase English letters from A to Z. This restriction makes the encoded representation unambiguous because decimal digits occur only as run counts.

The original input length must satisfy

$$1 \leq L \leq 50.$$

3. Buffer Organization

The program must reserve separate buffers for the original, encoded, and decoded strings.

A suitable organization is:

<code>original:</code>	<code>.space 52</code>	<i># Up to 50 letters, newline, and null byte</i>
<code>encoded:</code>	<code>.space 128</code>	<i># Worst case: A1B1C1... plus null byte</i>
<code>decoded:</code>	<code>.space 52</code>	<i># Up to 50 letters plus null byte</i>

All strings must be null terminated.

The encoded output must not overwrite the original input, and the decoded output must not overwrite either the original or encoded string.

4. Required Procedure Interfaces

encode_rle

Item	Requirement
Procedure name	encode_rle
Input	\$a0 contains the original-string address \$a1 contains the encoded-buffer address \$a2 contains the encoded-buffer capacity in bytes
Output	\$v0 contains the encoded length, excluding the null byte \$v1=0 on success \$v1=1 for invalid original input \$v1=2 for encoded-buffer overflow
Invocation	The procedure must be called using jal encode_rle
Return	The procedure must return using jr \$ra

decode_rle

Item	Requirement
Procedure name	decode_rle
Input	\$a0 contains the encoded-string address \$a1 contains the decoded-buffer address \$a2 contains the decoded-buffer capacity in bytes
Output	\$v0 contains the decoded length, excluding the null byte \$v1=0 on success \$v1=1 for a malformed encoded string \$v1=2 for decoded-buffer overflow
Invocation	The procedure must be called using jal decode_rle
Return	The procedure must return using jr \$ra

compare_strings

Item	Requirement
Procedure name	compare_strings
Input	\$a0 contains the first string address \$a1 contains the second string address
Output	\$v0=1 if the strings are identical \$v0=0 otherwise
Invocation	The procedure must be called using jal compare_strings
Return	The procedure must return using jr \$ra

5. Encoding and Decoding Algorithms

Encoding

Algorithm 2 `encode_rle`

Require: A non-empty null-terminated string containing only A–Z

Ensure: Produce its run-length encoded representation

```
1:  $i \leftarrow 0, j \leftarrow 0$ 
2: while  $input[i] \neq '\backslash 0'$  do
3:    $ch \leftarrow input[i]$ 
4:    $count \leftarrow 1$ 
5:   while  $input[i + count] = ch$  do
6:      $count \leftarrow count + 1$ 
7:    $encoded[j] \leftarrow ch$ 
8:    $j \leftarrow j + 1$ 
9:   Append every decimal digit of  $count$  in the correct order
10:   $i \leftarrow i + count$ 
11: Append the null terminator
12: Return  $j$ 
```

Decoding

Algorithm 3 `decode_rle`

Require: A valid null-terminated RLE string

Ensure: Reconstruct the original string

```
1:  $i \leftarrow 0, j \leftarrow 0$ 
2: while  $encoded[i] \neq '\backslash 0'$  do
3:   Read an uppercase letter  $ch$ 
4:   Advance  $i$ 
5:   if the next character is not a decimal digit then
6:     Return malformed-input status
7:    $count \leftarrow 0$ 
8:   while  $encoded[i]$  is a decimal digit do
9:      $count \leftarrow 10 \times count + (encoded[i] - '0')$ 
10:    Advance  $i$ 
11:   if  $count = 0$  then
12:     Return malformed-input status
13:   Append  $ch$  to the decoded buffer exactly  $count$  times
14: Append the null terminator
15: Return  $j$ 
```

6. Assignment Task

Write a MIPS32 assembly-language program that performs the following operations:

1. Ask the user to enter a non-empty uppercase string of at most 50 characters.
2. Read the input using syscall 8.
3. Remove the trailing newline inserted by syscall 8.
4. Validate that the resulting string contains between 1 and 50 characters.

5. Validate that every character is between A and Z.
6. Place the original-string address in `$a0`, the encoded buffer address in `$a1`, and the encoded-buffer capacity in `$a2`.
7. Call `encode_rle`.
8. Print the encoded string from `main`.
9. Place the encoded-string address in `$a0`, the decoded buffer address in `$a1`, and the decoded-buffer capacity in `$a2`.
10. Call `decode_rle`.
11. Print the decoded string from `main`.
12. Call `compare_strings` with the original and decoded strings.
13. Print whether the verification succeeded.
14. Print an appropriate error message when encoding or decoding fails.
15. Terminate the program using `syscall 10`.

7. Multi-Digit Count Requirement

The program must support run counts from 1 through 50.

When encoding a multi-digit count, the decimal digits must be written in the normal left-to-right order. For example,

12

must be written as the character sequence

'1', '2'

and not as

'2', '1'.

A temporary digit buffer, the runtime stack, or another correctly managed method may be used to reverse digits generated through repeated division by 10.

When decoding, the decimal count must be reconstructed using

$$count \leftarrow 10 \times count + digit.$$

8. Suggested Control-Flow Organization

```
.data
original: .space 52
encoded:  .space 128
decoded:  .space 52

# Input prompt
# Encoded-output message
# Decoded-output message
# Verification-success message
# Verification-failure message
```

```

    # Invalid-input message
    # Buffer-overflow message
    # Malformed-encoding message
    # Newline string

.text
.globl main

main:
    # Read and validate the original string
    # Remove the trailing newline
    # Call encode_rle
    # Print the encoded string
    # Call decode_rle
    # Print the decoded string
    # Call compare_strings
    # Print the verification result
    # Terminate using syscall 10

encode_rle:
    # Input:  $a0 = original string
    #         $a1 = encoded buffer
    #         $a2 = encoded-buffer capacity
    # Output: $v0 = encoded length
    #         $v1 = status code

    # Identify each maximal run
    # Write the character
    # Convert the run count to decimal text
    # Null terminate the encoded string

    jr $ra

decode_rle:
    # Input:  $a0 = encoded string
    #         $a1 = decoded buffer
    #         $a2 = decoded-buffer capacity
    # Output: $v0 = decoded length
    #         $v1 = status code

    # Read a letter
    # Parse one or more decimal digits
    # Repeat the letter count times
    # Null terminate the decoded string

    jr $ra

compare_strings:
    # Input:  $a0 = first string
    #         $a1 = second string
    # Output: $v0 = 1 if identical, otherwise 0

    jr $ra

```

The listing specifies only the required organization. Students must complete the MIPS32 instructions independently.

9. Input Validation and Error Conditions

The original input must satisfy all of the following:

- it must not be empty;
- its length must not exceed 50 characters; and
- every character must be an uppercase English letter from A through Z.

The decoder must reject encoded data containing:

- a count without a preceding uppercase letter;
- an uppercase letter not followed by at least one decimal digit;
- a zero count;
- an invalid character;
- a count that would overflow the decoded buffer; or
- a missing null terminator within the allocated buffer.

Messages similar to the following may be used:

```
Error: enter 1 to 50 uppercase letters only.  
Error: output buffer capacity exceeded.  
Error: malformed run-length encoded string.
```

10. Allowed Instructions and Directives

The solution may primarily use:

- `.data`, `.text`, `.globl`, `.ascii`, and `.space`;
- `la`, `li`, and `move`;
- `add`, `addi`, `addu`, `addiu`, `sub`, and `subu`;
- `mul`, or shift-and-add operations for multiplication by 10;
- `div` followed by `mflo` and `mfhi`;
- `sll`;
- `lb`, `lbu`, and `sb`;
- `lw` and `sw` when temporary word storage is used;
- `slt`, `slti`;
- `beq`, `bne`, `bgt`, `bge`, `blt`, `ble`, `bltz`, and `bgez`;
- `b`;
- `jal`;
- `jr`; and
- `syscall`.

Equivalent pseudo-instructions supported by MARS or QtSPIM may be used.

12. Sample Runs

Sample Run 1

```
Enter an uppercase string: AAAABBCCCCC
Encoded string: A4B2C5
Decoded string: AAAABBCCCCC
Verification successful: decoded string matches the original.
```

Sample Run 2

```
Enter an uppercase string: ABCDEF
Encoded string: A1B1C1D1E1F1
Decoded string: ABCDEF
Verification successful: decoded string matches the original.
```

Sample Run 3: Multi-Digit Count

```
Enter an uppercase string: AAAAAAAAAA
Encoded string: A12
Decoded string: AAAAAAAAAA
Verification successful: decoded string matches the original.
```

Sample Run 4: Invalid Input

```
Enter an uppercase string: AA11BB
Error: enter 1 to 50 uppercase letters only.
```

13. Required Test Cases

Original input	Expected encoded string	Expected decoded string
A	A1	A
AAAABBCCCCC	A4B2C5	AAAABBCCCCC
ABCDEF	A1B1C1D1E1F1	ABCDEF
AAAAAAAAAAAA	A12	AAAAAAAAAAAA
AABBBBBBBBBBCC	A2B10C2	AABBBBBBBBBBCC
ZZZZZZZZZZZZZZ	Z15	ZZZZZZZZZZZZZZ
ABABAB	A1B1A1B1A1B1	ABABAB
Empty input	Error message	Not required
ABC1	Error message	Not required
abc	Error message	Not required

Additional test cases may be used during evaluation.

14. Correctness Requirements

A correct implementation must satisfy the following conditions:

1. The original input is stored as a null-terminated byte string.
2. The trailing newline inserted by syscall 8 is removed.

3. The input length and character set are validated.
4. Every maximal run is identified correctly.
5. Every run character is written once to the encoded buffer.
6. Every run count is written in ordinary decimal notation.
7. Multi-digit counts are supported and written in the correct order.
8. The encoded string is null terminated.
9. The decoder requires an uppercase letter followed by one or more decimal digits.
10. Decimal counts are reconstructed correctly.
11. Zero counts and malformed encodings are rejected.
12. The decoded string is null terminated.
13. All buffer-capacity checks reserve space for the terminating null byte.
14. The original, encoded, and decoded strings occupy separate buffers.
15. `compare_strings` verifies the complete strings, including their lengths.
16. The procedures return results and status codes through the specified registers.
17. The final output is printed by `main`.
18. The program terminates using syscall 10.

15. Submission Requirement for Question 2

The source file for Question 2 must be named:

`rle_codec_{Group_Number}.S`

The file must begin with a comment header similar to:

```
# -----
# Group Number:
# Group Members (Name and Roll Number):
# 1.
# 2.
# 3.
# Course: Computer Organization and Architecture Laboratory
# Assignment: MIPS32 Assignment 6
# Question: 2
# Problem: Run-length encoding, decoding, and verification
# Date:
# -----
```

Common Submission Requirements

The assignment must be submitted as a group submission through Moodle. Only one submission is required from each group.

Submit a single zipped folder named:

`MIPSAssignment6_{Group_Number}.zip`

The folder must contain the following files:

File	Contents
<code>postfix_evaluator_{Group_Number}.S</code>	Solution for Question 1: Postfix Expression Evaluation Using an Explicit Stack
<code>rle_codec_{Group_Number}.S</code>	Solution for Question 2: Run-Length Encoding and Decoding